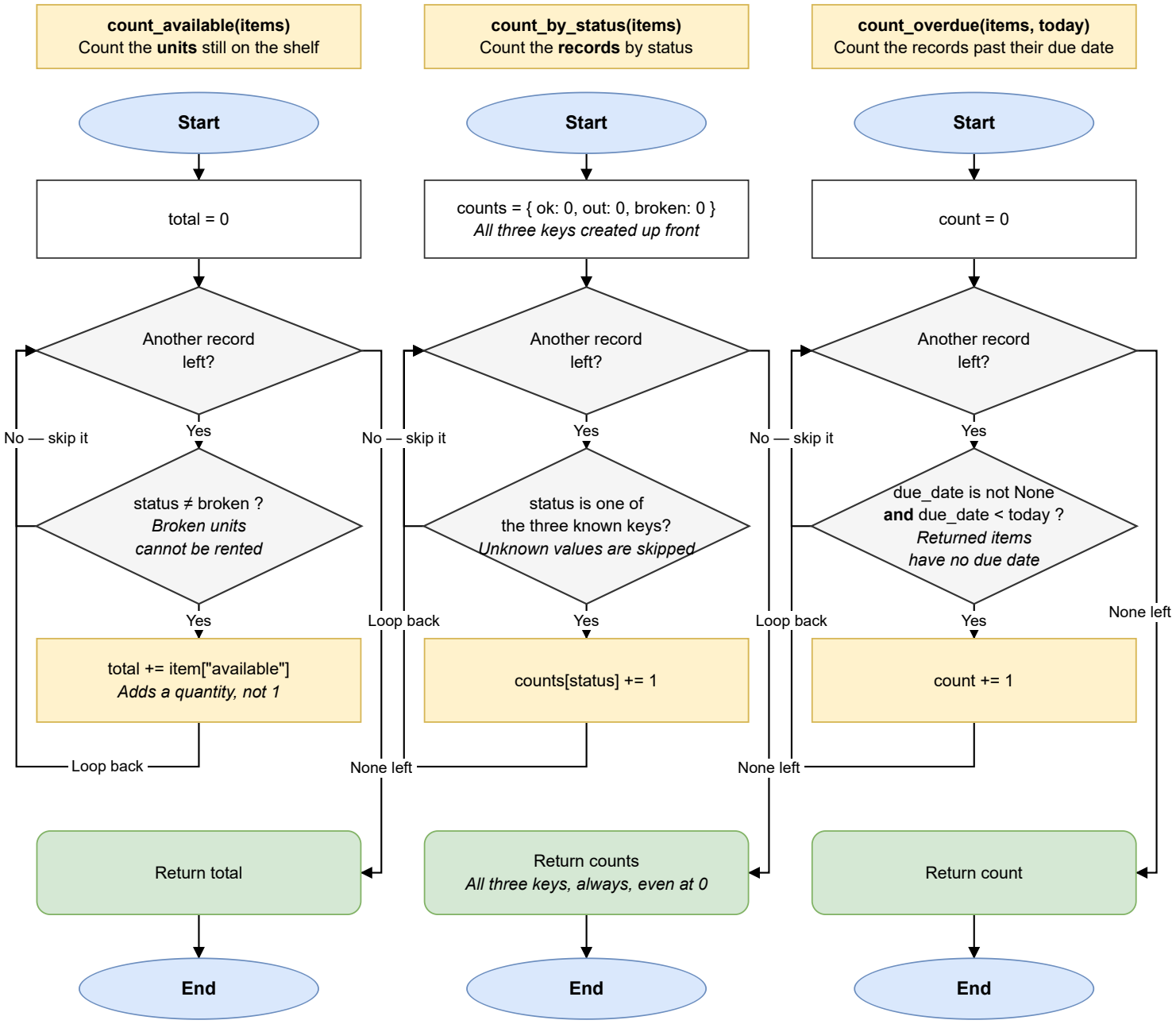
Flowchart 5 — Counting: count_available · count_by_status · count_overdue

search.py · All three are a single pass over the data — the same skeleton, differing only in the condition and in what is added



Why all three are O(n), with no shortcut

- Counting differs from searching: there is **no "right one" to stop at**. Every item must be examined before the number is known, so none can be skipped.
- Binary search does not apply — halving needs one target that can be compared as greater or smaller, and counting has no such target.
- A hash table does not help either: it answers "where is this one", never "how many satisfy the condition".
- **Memory is O(1) for all three** — count_by_status holds three fixed slots, which do not grow with n.
- count_by_status produces all three numbers in a **single pass** rather than three separate loops, reading the data three times less.
- Dates compare correctly as plain strings because YYYY-MM-DD runs from the largest unit to the smallest; stored as DD/MM/YYYY this stops being true at once.